UML

CLASS DIAGRAMS AND DEPENDENCIES

What is UML?

Unified Modeling Language is used to visually represent the structure and design of a software system before implementing it in code.

A UML class diagram represents classes, their attributes, methods, and relationships.

UML Class Structure

A class is represented by a rectangle divided into three sections:

Chicken
- name : String - age : int - isMolting : boolean
+ doStuff(forTime : int) - cluck() - layAnEgg() : Egg

Class name: identifies the class.
Attributes: describe the data stored by the class.
Methods: describe the actions the class can perform.

Visibility Symbols

Symbol	Meaning
+	Public
-	Private
#	Protected

Attribute and method notation

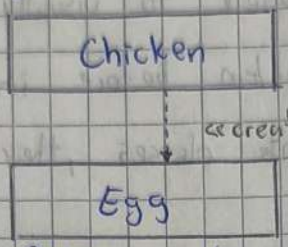
- name: String — attribute name and data type
- doStuff(forTime: int) — method name and parameter
- layAnEgg(): Egg — method that returns an Egg object

What is a Dependency?

A dependency indicates that one class's definition or implementation depends on another class.

UML notation

A dependency is represented by a dashed line with an arrow pointing toward the class being used.



Chicken creates an Egg

The arrow starts at the dependent class and points toward the class it depends on.

Types of Dependencies

Label	Meaning	Example
«use»	The first class uses the second class in a data member definition or as a method parameter	ChickenCoop uses Chicken
«create»	The first class creates objects of the second class.	Chicken creates Egg.
«delete»	The first class deletes objects of the second class.	A class deletes an object of another class.

Naming Conventions

- Class names — UpperCamelCase : ChickenFarmer
- Attribute names — lowerCamelCase : isMolting
- Method names — lowerCamelCase : doStuff()

OBJECT ORIENTATION

GOOD PRACTICES

Conceptual Modeling

Conceptual modeling is the process of representing ideas and concepts in a form that helps people understand, analyze, design, and communicate a system.

Why do we use conceptual modeling?

- To better understand a real-world system
- To visualize the system as it currently exists
- To describe how the system should become.
- To document important design decisions.
- To create a plan that guides software construction.
- To comunicate ideas efficiently.

Conceptual modeling can be used throughout the software development process, including analysis, specification, design, implementation, and deployment.

UML and Conceptual Modeling

UML provides different diagram types for representing different aspects of a system.

- Use case diagrams: Describe who needs the system and their goals.
- Class diagrams: Describe classes and their relationships.
- Interaction diagrams: Describe how objects communicate
- State charts: Describe the internal behavior of objects or methods.

Loose Coupling and Layers

Loose coupling means reducing unnecessary dependencies between software components.

Each layer should provide a well-defined abstraction to higher-level layers and may use components in its own layer or abstractions from lower-level layers.

Aggregation vs. Inheritance

Aggregation	Inheritance
Object A contains or uses object B Represents a has-a relationship Can be changed at runtime, making it flexible	Class A inherits from class B Typically represents an is-a relationship Provides the capabilities of the parent class to the child class.

The recomendation is to prefer aggregation over inheritance when all other considerations are equivalent. Composition is a stronger form of aggregation

Programming to an Interface or Abstraction

An interface defines the operations or properties available to users of a component.

Programming to an interface or abstraction helps reduce unnecessary dependencies on specific implementations and manage coupling.

Meaningful Identifiers

- Class names nouns or noun phrases.
- Variable names describe the purpose of the object, typically lowerCamelCase.
- Method names describe actions, such as computeCost().

Unit testing

Unit testing involves testing individual units of code using executable test cases.

Benefits include finding design errors, verifying correctness, documenting how methods and classes should be used, and supporting regression testing.

Tema | Día | Mes | Año | ESTILO

INTRODUCTION TO UML

What is UML?

Unified Modeling Language is a modeling language used to visualize, specify, construct, and document software systems.

It supports different development processes, incremental modeling, and customization. UML is not a programming language.

UML Building Blocks

- Structural elements: objects, classes, relationships, interfaces, components, and nodes.
- Behavioral elements: messages, states, transitions, and events.
- Organizational elements: packages and views.
- Annotations: comments and specifications.
- Extension elements: templates and stereotypes.

Modeling elements are like words, diagrams are like sentences, and models are like complete books.

The Three Modeling Perspectives

Perspective	Purpose	Focus
Analysis	Understanding	Real-word concepts and the problem domain
Specification	Solving and planning	What the system should become/responsabilities.
Implementation	Construction	Software objects, components, and implementation details.

Structural and Behavioral Diagrams

Structural diagrams

- Class diagrams
- Object diagrams
- Component diagrams
- Deployment diagrams

Behavioral diagrams

- Use case diagrams
- Sequence diagrams
- Communication diagrams
- State chart diagrams
- Activity diagrams.

Structural diagrams describe the organization of a system

Behavioral diagrams describe how the system behavers, how objects interact, or how activities and states changes.

Diagram definitions

- Class diagrams: show classes, their properties, and their relationships.
- Object diagrams: show objects and their relationships at a particular point or in a specific example.
- Component diagrams: shows software components and their relationships.
- Deployment diagrams: show nodes and their relationships in a system architecture.
- Use case diagram: shows actors and use cases.
- Sequence diagram: emphasizes the time order of messages between objects.
- Communication diagram: emphasizes the structural organization of objects that comunicate
- Statechart diagram: show states, transitions, events, and activities.
- Activity diagram: shows the flow and ordering of activities.

Views and Abstraction

Use case view
Logical view
Implementation view
Process view
Deployment view

Tema | Día | Mes | Año | ESTILO

BASIC USE CASES

Actors and Use Cases

Actor: An external entity that interacts with a system. Examples: users, other software system, and hardware devices.

Use case: Represent a goal that an actor wants to achieve using the system.

Use case diagram: Visually represents actors, use cases, relationships, and the system boundary.

All users are actors, but not all actors are users.

USE CASES	SYSTEM INTERACTIONS
Answer What and Why?	Answer How?
Describe user goals.	Describe interaction steps.
Example: Alter student Grade	Select a course, enter a grade and confirm

Use case Relationships

«include» Incorporates required behavior from another use case

«extend» Adds optional or conditional behavior to another use case

Generalization Represents a specialized version of a general use case.

Use cases help identify system functionality, understand user needs, plan development, reuse behavior, and design system tests.

Notas

CLASS DIAGRAMS

Object: An individual entity with identity, state and/or behavior.

Class: An abstraction representing objects that share common characteristics.

Associations

An association is a structural relationship between objects of diferent classes. Navigation can be unidirectional, bidirectional, or unspecified.

Generalization

Generalization represents a relationships between a general class and a more specialized class. For example, student can specialize Person

Aggregation and Composition

Aggregation A whole-part relationship represented by a hollow diamond.

Composition A stronger whole-part relationship represented by a filled diamond. The lifetime of the part is tied to the lifetime of the whole

Multiplicity

Multiplicity defines how many objects can participate in a relationship

Notation	Meaning
1	Exactly one
0..1	Zero or one
*	Many, possibly zero
1..*	One or more
0..8	Between zero and eight

Class Diagram Perspectives

- Analysis: Represents real-word concepts and objects
- Specification: Describes software objects and their responsibilities
- Implementation: Represents software classes and implementation details.